

Module 03 — Jinja Basics for dbt

Tier: ● Beginner · Duration: 75 min · Prerequisites: Module 02

Why this module exists: You've been seeing `{{ ref() }}`, `{{ source() }}`, `{{ config() }}`, and `{% if %}` blocks since Module 01. Without a proper grounding in Jinja syntax, it's easy to pattern-match without really understanding what's happening. This module gives you just enough Jinja to read and write any standard dbt model confidently — no macro programming, no advanced dispatch.

Agenda

Time	Duration	Topic	Learning Goal	Mode	Participant Activity	Materials	Trainer Notes
00:00	10 min	Recap Module 02	Confirm last session before new content	Q&A	Answer from memory	—	Ask all 4 prep questions cold. Specifically probe: "at which phase does a Jinja error appear?" — this connects directly to today's content
00:10	15 min	What Jinja is and how dbt uses it	Understand that dbt models are templates, not plain SQL	Present	Annotate diagram	This doc	Key mental model: dbt reads your .sql file, runs it through Jinja, produces plain SQL, then sends that to Snowflake. The Jinja never reaches Snowflake.

00:25	20 min	The four Jinja constructs you need	Know <code>{{ }}</code> , <code>{% %}</code> , <code>{# #}</code> , and whitespace control	Present + live code	Follow along in editor	This doc	Write each example live. Deliberately use wrong delimiters and show the error.
00:45	15 min	dbt's built-in Jinja functions	Read and write <code>ref()</code> , <code>source()</code> , <code>config()</code> , <code>var()</code>	Present + live code	Annotate own copy	This doc	These are the only four you need for all Beginner modules. Don't introduce macros yet — that's Module 09.
01:00	20 min	Exercise: read and write models	Read compiled output, write a model with correct Jinja	Practice	Solo exercise	Exercise below	Circulate. The most common error: using <code>{{ }}</code> for an <code>{% if %}</code> block. Watch for it.
01:20	5 min	Debrief + prep questions	Consolidate	Debrief	Verbal	—	—

Content

Part A — What Jinja Is

Jinja is a templating language. dbt uses it to add logic to SQL files that plain SQL can't express.

When dbt runs, it:

1. Reads your `.sql` file
2. Processes it through the Jinja engine → produces plain SQL
3. Sends that plain SQL to Snowflake

The Jinja syntax never reaches Snowflake. What Snowflake receives is always plain SQL.

You can always see what Snowflake received at: `target/compiled/analytics/models/...`

Part B — The Three Delimiter Types

Delimiter	Purpose	Example
<code>{{ }}</code>	Expression — outputs a value	<code>{{ ref('dim_patient') }}</code>
<code>{% %}</code>	Statement — logic, no output	<code>{% if target.name == 'prod' %}</code>
<code>{# #}</code>	Comment — ignored entirely	<code>{# TODO: add grain doc #}</code>

The most common beginner mistake: using `{{ }}` for an `if` block. This trips people up because the syntax looks similar. But the delimiter determines whether Jinja executes the statement or tries to output it as a value.

```
-- ❌ WRONG — this will output the text of the condition, not execute it
{{ if is_incremental() }}
```

```
-- ✅ CORRECT — statement delimiter, no output
{% if is_incremental() %}
```

Part C — The Four dbt Jinja Functions You Need

1. `{{ ref() }}` — reference another model

```
-- Without ref() — hardcoded, brittle
SELECT *
FROM SILVER.APPOINTMENTS.dim_patient

-- With ref() — dbt resolves this to the correct schema based on target
SELECT *
FROM {{ ref('dim_patient') }}
```

What `{{ ref('dim_patient') }}` compiles to in dev:

```
FROM SILVER_DEV.TESTING__dev_jane.dim_patient
```

What it compiles to in prod:

```
FROM SILVER.PUBLIC.dim_patient
```

This is why you never hardcode table names. `ref()` makes your models environment-aware and adds the dependency to the DAG.

2. `{{ source() }}` — reference a raw source table

```
SELECT *
FROM {{ source('hubspot', 'contacts') }}
```

Compiles to:

```
FROM BRONZE.HUBSPOT.contacts
```

Sources are declared in `sources.yml`. `source()` registers the table as a DAG node so freshness checks and lineage work. This is covered in depth in Module 05.

3. `{{ config() }}` — configure a model inline

```
{{ config(
    materialized = 'incremental',
    unique_key   = 'contact_key',
    on_schema_change = 'sync_all_columns'
) }}

SELECT ...
```

`config()` overrides whatever is set in `dbt_project.yml` for this specific model. Common overrides:

- `materialized` — `view`, `table`, `incremental`, `ephemeral`
- `unique_key` — required for incremental models
- `tags` — for selective runs
- `alias` — to name the output table differently from the file name

4. `{{ var() }}` — access a runtime variable

```
WHERE created_at >= '{{ var("start_date") }}'
```

Variables are defined in `dbt_project.yml` or passed at runtime:

```
dbt run --vars '{"start_date": "2024-01-01"}'
```

Use `var()` for environment-specific values, date ranges, or feature flags. Don't use it for connection details — that's what `profiles.yml` is for.

Part D — Whitespace Control

Jinja adds newlines. Use `-` to strip them when it matters for readability:

```
{%- if is_incremental() -%}
    AND updated_at > (SELECT MAX(updated_at) FROM {{ this }})
{%- endif -%}
```

The `-` inside `{%-` and `-%}` trims whitespace before/after the tag. This only affects how the compiled SQL looks — it has no effect on execution.

`target.name` — environment-conditional logic

The `target` object exposes the active profile target (`dev`, `prod`, etc.). The most useful property is `target.name`:

```
SELECT *
FROM {{ ref('fct_appointments') }}
{% if target.name != 'prod' %}
    WHERE appointment_date >= DATEADD('month', -3, CURRENT_DATE())
{% endif %}
```

In dev this adds a 3-month filter — cheaper and faster. In prod the filter is removed and the full dataset is scanned. Note: this is a `{% %}` statement, not a `{{ }}` expression.

This pattern is especially common inside incremental models to limit dev scans. You won't need it often in the Foundations tier, but you'll see it in the project codebase.

Part E — `{{ this }}` — reference the current model's table

```
{% if is_incremental() %}
    WHERE updated_at > (SELECT MAX(updated_at) FROM {{ this }})
{% endif %}
```

`{{ this }}` refers to the table this model materialises to. It's used almost exclusively in incremental models to filter only new records.

Exercise (20 min)

Project context: This exercise starts your staging layer. By the end you'll have one working staging model.

Task 1 — Predict compiled output before writing anything

Given this model, write what the compiled SQL will look like for the **dev** target before running anything:

```
{{ config(materialized='view') }}

SELECT
    c.contact_id,
    c.email,
    p.pipeline_name
FROM {{ source('hubspot', 'contacts') }} c
LEFT JOIN {{ ref('dim_pipeline') }} p
    ON c.pipeline_id = p.hubspot_pipeline_id
```

What does `{{ source('hubspot', 'contacts') }}` compile to? What does `{{ ref('dim_pipeline') }}` compile to? What happens to the `{{ config() }}` block?

► Expected compiled SQL (dev target)

Task 2 — Write `stg_hubspot__contacts.sql`

Create `models/staging/hubspot/stg_hubspot__contacts.sql`.

The Bronze source (`BRONZE.HUBSPOT.contacts`) has: `contact_id` , `email` , `pipeline_id` , `_ingested_at` .

Requirements:

- Materialise as a `view`
- Reference the source via `{{ source('hubspot', 'contacts') }}`
- Select all four columns; rename `_ingested_at` → `ingested_at`

Then run:

```
dbt compile --select stg_hubspot__contacts
```

Open `target/compiled/analytics/models/staging/hubspot/stg_hubspot__contacts.sql` . Confirm that `{{ source('hubspot', 'contacts') }}` compiled to `BRONZE.HUBSPOT.contacts` .

► Expected model

Reference Material

- [Jinja2 docs — template syntax](#)
 - [dbt Jinja functions reference](#)
 - [dbt `ref\(\)` docs](#)
 - [dbt `source\(\)` docs](#)
-

Prep Questions for Module 04

1. What does `{{ ref('dim_patient') }}` compile to in your dev environment?
2. What's the difference between `{{ }}` and `{% %}` ?
3. When would you use `{{ config() }}` instead of setting materialisation in `dbt_project.yml` ?
4. What does `{{ this }}` refer to?